AI LAB D	Wierzchoń Oktawiusz Album 55638 Grupa 2	**Wersja 2026-09-14**

REST API CLIENT

SPIS TREŚCI

Spis treści 1
Cel zajęć 1
Rozpoczęcie 1
Uwaga 1
Wymagania 2
Badanie API 3
Implementacja 4
Commit projektu do GIT 7
Podsumowanie 8

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- pobieranie danych z zewnętrznych zasobów za pomocą REST API
- zdobywanie wiedzy na temat zewnętrznych API za pomocą dokumentacji typu Swagger
- wysyłanie asynchronicznych żądań z wykorzystaniem XMLHttpRequest i Fetch API

W praktycznym wymiarze uczestnicy stworzą dynamiczną stronę HTML pozwalającą na wyświetlanie bieżącej informacji pogodowej oraz prognoz dla zadanej przez użytkownika miejscowości.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie wykonywania połączeń synchronicznych i asynchronicznych z poziomu JS na stronie.

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w poprzednio, wejdź do właściwości dokumentu i zaktualizuj niestandardowe pola.

WYMAGANIA

Szablon plików wymaganych w LAB D został dostarczony razem z instrukcją.

W ramach LAB D przygotowane powinny zostać:

- pojedyncza strona HTML ze skryptem ładowanym z zewnętrznego pliku JS i stylami z zewnętrznego pliku CSS;
- pole tekstowe (input typu „text”) do wprowadzania adresu
- przycisk „Pogoda”, po kliknięciu którego wykonywane jest zapytanie asynchroniczne:
    - do API Current Weather: https://openweathermap.org/current za pomocą XMLHttpRequest
    - do API 5 day forecast: https://openweathermap.org/forecast5 za pomocą Fetch API
- obsługa zwrotki z obu API – wypisanie pogody bieżącej oraz prognoz poniżej pola wyszukiwania.
- wymagane jest użycie własnego klucza do API, w przypadku użycia cudzego klucza praca nie zostanie oceniona
- aplikacja musi być responsywna oraz różnić się wyglądem od zaprezentowanego poniżej, w przypadku wykorzystania poniższego wyglądu praca nie zostanie oceniona



Wygeneruj klucz do API. Ponieważ aktywacja może chwilę potrwać, na czas trwania laboratorium możesz wykorzystać „służbowy” klucz: `7ded80d91f2b280ec979100cc8bbba94`. **UWAGA!** Klucz zostanie dezaktywowany niedługo po zajęciach. Musisz wygenerować swój własny.

Prowadzący omówi powyższe wymagania. Upewnij się, czy wszystko rozumiesz.

W przypadku blokady twórczej można posiłkować się filmem: https://www.youtube.com/watch?v=WoKp2qDFxKk jednakże spróbuj rozwiązać ten problem samodzielnie! Wygląd aplikacji musi różnić się od prezentowanego na filmie!

Tu umieść swoje notatki:

…notatki…

Badanie API

Poświęć kilka minut na wykonanie przykładowych zapytań do API z poziomu pasku adresu przeglądarki. Podaj wymagane parametry dla osiągnięcia różnych wyników. Zbadaj odpowiedzi API, aby uzyskać pełen obraz wymagań i możliwości API.

Wstaw zrzut ekranu zawierający Twój własny klucz na stronie https://www.openweathermap.org (**zakładka API keys)**



oraz **fragment kodu,** w którym wykorzystany zostaje Twój **własny klucz**

```
const API_KEY : string  = "1f0903f34f430449f859c9d6a1e09490";
```

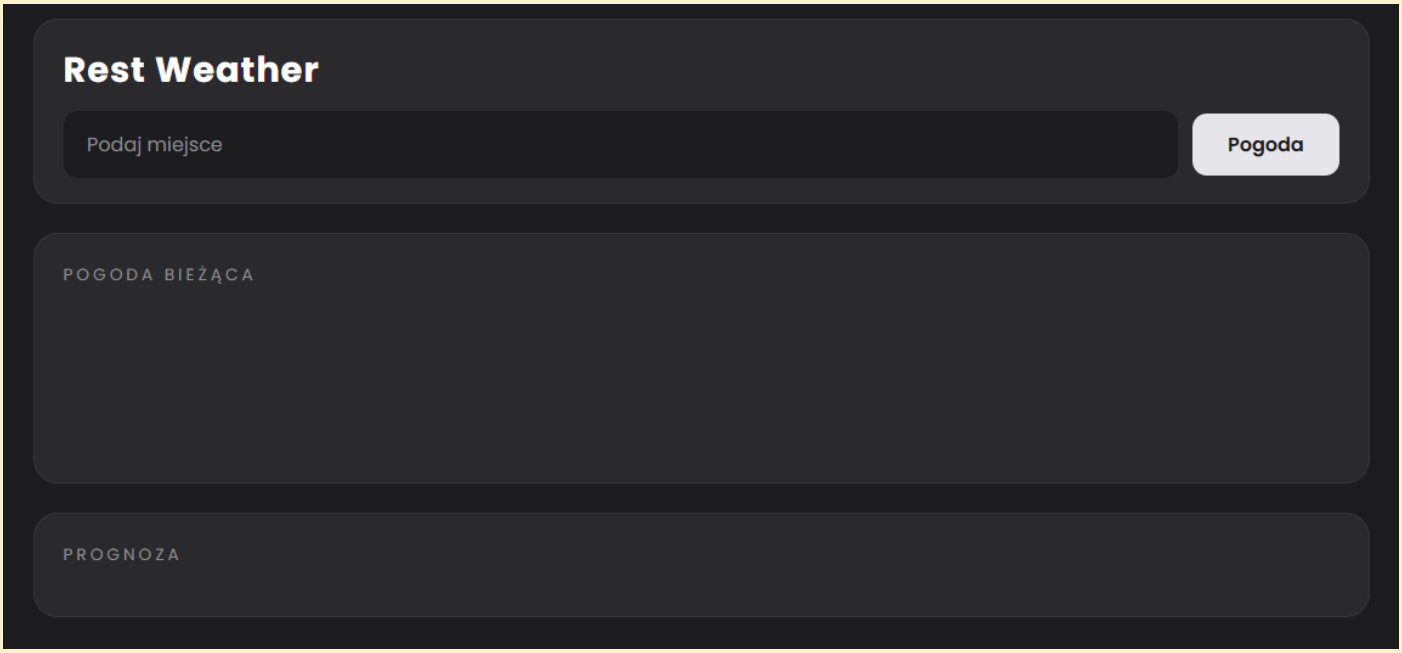
```
function getCurrent(city) : void  {
  const xhr : XMLHttpRequest  = new XMLHttpRequest();
  const url : string  = `https://api.openweathermap.org/data/2.5/weather?q=${encodeURIComponent(city)}&appid=${API_KEY}&units=metric&lang=pl`;
  xhr.open( method: "GET", url,  async: true);
```

```
function getForecast(city) : void  {
  const url : string  = `https://api.openweathermap.org/data/2.5/forecast?q=${encodeURIComponent(city)}&appid=${API_KEY}&units=metric&lang=pl`;
  const box : HTMLElement  = document.getElementById( elementId: "forecastList");
```

IMPLEMENTACJA

Tradycyjnie implementację należy zacząć od zbudowania w HTML + CSS wszystkich wymaganych elementów / placeholderów na te elementy. Następnie krok po kroku należy implementować poszczególne zachowania.

Wstaw zrzut ekranu zawierającego stronę ze wszystkimi elementami, tj. pole tekstowe, przycisk, miejsce do wyświetlenia pogody i prognozy:

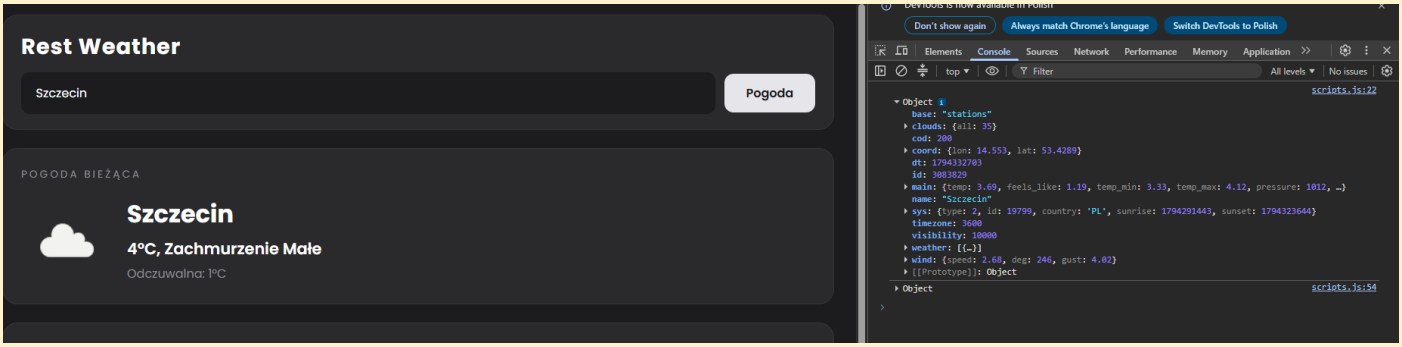


Punkty:	0	1

Wstaw zrzut ekranu kodu odpowiedzialnego za wysyłanie żądania do `current` za pomocą `XMLHttpRequest`:

```
function getCurrent(city) :void {
  const xhr :XMLHttpRequest = new XMLHttpRequest();
  const url :string = `https://api.openweathermap.org/data/2.5/weather?q=${encodeURIComponent(city)}&appid=${API_KEY}&units=metric&lang=pl`;
  xhr.open( method: "GET", url, async: true);
  xhr.onload = () :void => {
    // [Claude Opus 5.5] zmiana: komunikaty trafiają do #currentWeather zamiast #current,
    // żeby nie nadpisywać nagłówka "Pogoda bieżąca"
    const box :HTMLElement = document.getElementById( elementId: "currentWeather");
    if (xhr.status === 200) {
      const data = JSON.parse(xhr.responseText);
      console.log(data);
      showCurrent(data);
    } else if (xhr.status === 401) {
      // [Claude Opus 5.5] dodane: osobny komunikat dla błędnego / jeszcze nieaktywnego klucza API
      box.textContent = "Błędny lub jeszcze nieaktywny klucz API";
    } else {
      box.textContent = "Nie znaleziono miejscowości";
    }
  };
  // [Claude Opus 5.5] dodane: obsługa błędu sieci (brak internetu, zablokowane żądanie)
  xhr.onerror = () :void => {
    document.getElementById( elementId: "currentWeather").textContent = "Błąd połączenia";
  };
  xhr.send();
}
```

Wstaw zrzut ekranu pokazujący otrzymaną odpowiedź za pomocą `console.log()` w przeglądarce.

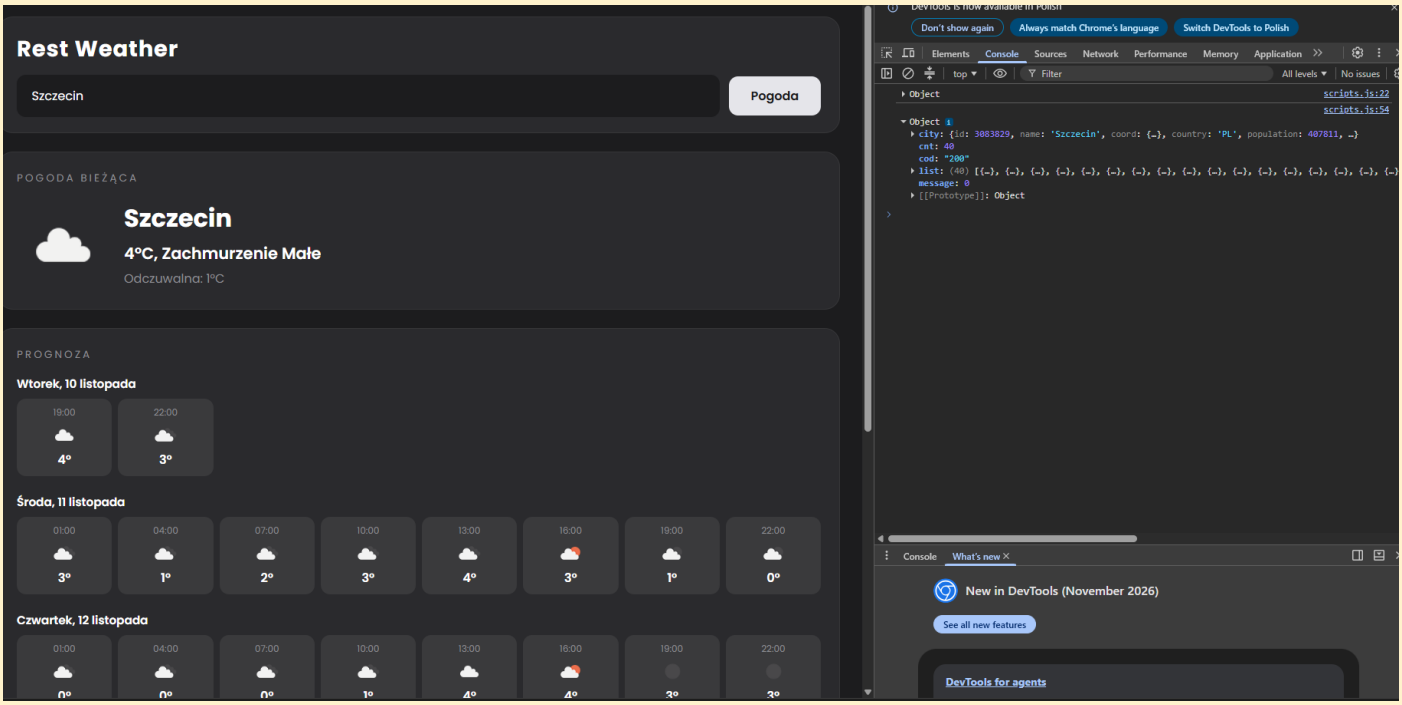


Punkty:	0	1

Wstaw zrzut ekranu kodu odpowiedzialnego za wysyłanie żądania do `forecast` za pomocą `fetch`:

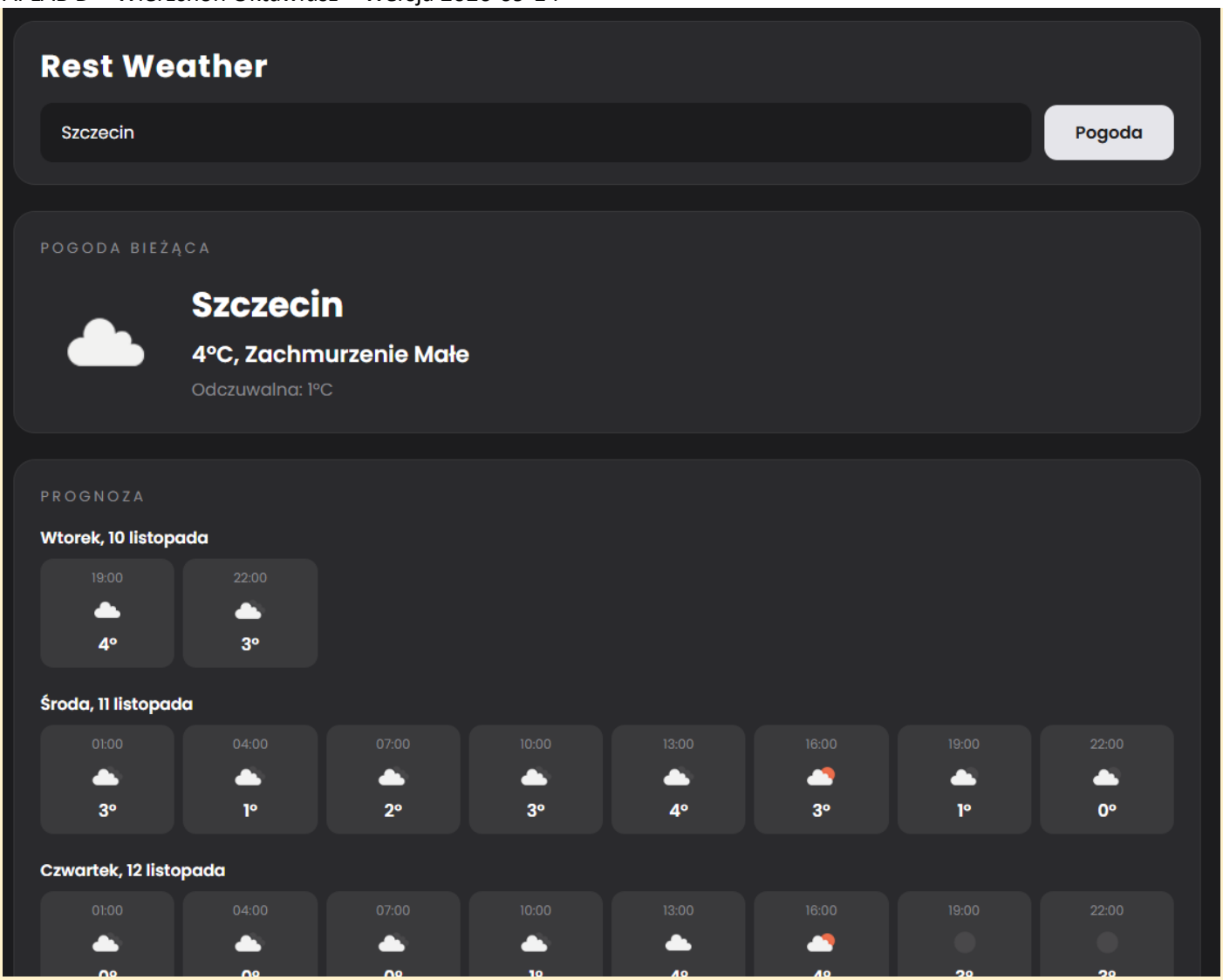
```
// [Claude Opus 5.5] zmiana: XMLHttpRequest zastąpiony przez Fetch API
Show usages
function getForecast(city) : void  {
  const url : string  = `https://api.openweathermap.org/data/2.5/forecast?q=${encodeURIComponent(city)}&appid=${API_KEY}&units=metric&lang=pl`;
  const box : HTMLElement  = document.getElementById( elementId: "forecastList");
  fetch(url) Promise<Response>
    .then((response : Response ) => {
      if (response.status === 401) {
        throw new Error("Błędny lub jeszcze nieaktywny klucz API");
      }
      // fetch nie odrzuca obietnicy przy statusach 4xx/5xx, trzeba sprawdzić je ręcznie
      if (!response.ok) {
        throw new Error("Nie znaleziono miejscowości");
      }
      return response.json();
    }) Promise<any>
    .then((data) : void  => {
      console.log(data);
      showForecast(data);
    }) Promise<void>
    .catch((error) : void  => {
      if (error instanceof TypeError) {
        box.textContent = "Błąd połączenia";
      } else {
        box.textContent = error.message;
      }
    });
}
```

Wstaw zrzut ekranu pokazujący otrzymaną odpowiedź za pomocą `console.log()` w przeglądarce.



Punkty:	0	1

Wstaw zrzut ekranu przedstawiającego wizualizację prognoz pogody:



Upewnij się, że widoczne są pasek wyszukiwania ze wskazaną miejscowością, a także zarówno pogoda bieżąca jak i prognozy pogody.

Punkty:	0	1

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało. Upewnij się, że aktualna wersja rozwiązania jest dostępna przez GitHub Pages.

Podaj link do swojego repozytorium GitHub Pages:
Repo:
https://github.com/Oktavviusz/oktavviusz.github.io
Pages:
https://oktavviusz.github.io/lab-d/

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. Nie używaj LLM.
Fajne zadanie.
…podsumowanie…

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.